

Architectural Paradigms for Real-Time, Local-First ASR Post-Processing: A Deep Dive into WhisperLocal

Executive Overview: The "Last Mile" of Local Speech Recognition

The democratization of Automatic Speech Recognition (ASR) through open-weights models like OpenAI's Whisper has fundamentally altered the landscape of human-computer interaction. We have transitioned from a paradigm where high-accuracy transcription was the exclusive domain of cloud giants—Google, Amazon, and Microsoft—to one where consumer-grade hardware can perform inference with near-human parity. `whisper.cpp`, a high-performance C/C++ port of the Whisper architecture, epitomizes this shift, enabling efficient, offline execution on standard Windows environments using AVX instructions and modest GPU acceleration.

However, raw transcription is merely the foundation, not the complete structure, of a professional dictation system. While Whisper excels at phonetic decoding, it is fundamentally a generalist. It lacks the semantic rigidity required for specialized domains such as radiology, commercial litigation, or software development. It does not inherently know that a "myocardial infarction" requires specific formatting in a medical report, nor does it understand that "def function" implies a specific syntactic structure in Python. Furthermore, raw ASR output is static; it does not learn from user corrections, leading to the frustrating "repeat error" phenomenon where a user must fix the same misinterpretation of their name or a project code daily.

This report addresses the "last mile" engineering challenge: transforming raw text streams into structured, domain-accurate content in real-time. The "WhisperLocal" project aims to build a post-processing pipeline that rivals cloud-based adaptive systems (like Nuance Dragon or Otter.ai) while operating under strict offline constraints. The core engineering challenges identified are latency (processing text in under 200ms to maintain flow), scalability (managing massive domain dictionaries), adaptability (learning from user edits), and context (understanding entities across transcription chunks).

The following analysis proposes a **Cascaded Hybrid Architecture**. This architecture rejects the notion of a single monolithic model for post-processing. Instead, it advocates for a tiered approach: utilizing ultra-fast statistical methods for immediate gating, quantized static embeddings for semantic disambiguation, and offline Large Language Models (LLMs) for asynchronous rule generation. This ensures that the real-time inference loop remains

lightweight (< 200 ms) while the system retains the capacity for deep personalization and continuous learning.

1. The Latency Budget: Psychoacoustics and System Constraints

To architect a solution for WhisperLocal, one must first rigorously define the performance envelope. The constraints of a "real-time" dictation app are dictated not just by hardware limits, but by human perception.

1.1 The 200ms Threshold and Cognitive Load

Research in Human-Computer Interaction (HCI) has long established that the delay between a user's action (speech) and the system's response (text appearing on screen) significantly impacts cognitive load. Jakob Nielsen's classic response time limits suggest that:

- **< 100 ms** feels instantaneous.
- **100 ms – 1000 ms** is noticeable but maintains the user's flow of thought.
- **> 1000 ms** breaks the user's attention, leading to a context switch.

In the context of dictation, the "action" is the completion of an utterance. However, whisper.cpp itself consumes a significant portion of this budget. On a standard consumer CPU (e.g., Intel Core i7 or AMD Ryzen 7 without a dedicated high-end GPU), decoding a 30-second audio chunk can take 500ms to several seconds depending on the model size (Base vs. Large). WhisperLocal utilizes streaming or chunked inference to mitigate this, but the post-processing pipeline operates *after* the text is decoded.

If the ASR inference takes 1500ms and the post-processing adds another 500ms, the total latency approaches 2 seconds—a "sluggish" experience that feels disconnected from the speaker's voice. Therefore, the post-processing pipeline—comprising domain detection, formatting, and correction—is allocated a strict budget, defined in the user query as **< 200 ms**. This is a "hard real-time" constraint in the context of user experience. If the domain detector takes 300ms, the system feels broken.

1.2 The Local Compute Constraint

The requirement for **100% local execution** on Windows imposes severe architectural limits. We cannot offload the heavy lifting to a cloud-based GPT-4 API. We must rely on the resources available on the user's machine, which are likely already taxed by the ASR model.

- **Memory Bandwidth:** Running a Large-v3 Whisper model requires significant VRAM/RAM bandwidth. The post-processing models must be small enough to stay in the CPU cache or have a minimal memory footprint to avoid thrashing.

- **Thread Contention:** whisper.cpp is heavily multi-threaded. The post-processing pipeline should ideally be single-threaded or use very few threads to avoid context-switching overhead that degrades the ASR performance.
- **Instruction Sets:** Approaches must utilize optimized libraries (like ONNX Runtime) that leverage AVX2/AVX-512 instructions on the CPU to maximize throughput per cycle.

2. Real-Time Automatic Domain Detection

The first and most critical stage of the intelligent pipeline is **Automatic Domain Detection**. The system must analyze a buffered transcript (10 to 500 words) and classify it into a predefined domain (e.g., Medical, Legal, Programming, General) to activate the correct regex profile.

This classification must happen in real-time. The research indicates that a "one size fits all" model is inappropriate here. A massive Transformer (like BERT) is too slow; a simple keyword match is too dumb. The optimal solution is a **Cascaded Classifier**.

2.1 Level 1: Statistical Gatekeeping (TF-IDF & Bloom Filters)

At the base of the cascade lies the fastest possible method: statistical analysis of keyword frequency. This layer acts as a "Gatekeeper." If the signal is strong enough here, we bypass the more expensive layers entirely.

2.1.1 Term Frequency-Inverse Document Frequency (TF-IDF)

TF-IDF is a numerical statistic that reflects how important a word is to a document in a collection or corpus. In the context of WhisperLocal, the "document" is the current transcript buffer, and the "corpus" is the aggregate of all potential domains.

Mathematical Foundation:

For a term t in a transcript d :

$$\text{TF}(t, d) = \frac{\text{count}(t, d)}{\text{total words in } d}$$

$$\text{IDF}(t) = \log \left(\frac{\text{Total Domains}}{\text{Number of Domains containing } t} \right)$$

$$\text{TF-IDF} = \text{TF} \times \text{IDF}$$

This method penalizes words that appear everywhere (e.g., "the", "meeting", "email") and boosts words unique to a specific domain (e.g., "laparoscopy" for Medical, "plaintiff" for

Legal).

- **Latency Profile:** The inference involves a hash map lookup for each word in the buffer and a dot product of sparse vectors. On modern CPUs, this executes in **microseconds (< 1ms)**.¹
- **Implementation Strategy:**
 - **Pre-computation:** During the "offline" phase (when profiles are built), the system computes the IDF vectors for each domain.
 - **Runtime:** As text arrives, update the TF vector of the sliding window. Compute the cosine similarity between the current TF-IDF vector and the pre-computed Domain Vectors.
 - **Thresholding:** If the similarity score for "Medical" exceeds a high confidence threshold (e.g., 0.85), immediately switch to the Medical profile.

2.1.2 Bloom Filters for Deterministic Triggers

For certain domains, specifically **Programming/Code Mode**, probabilistic analysis is unnecessary. If a user dictates specific reserved keywords, the intent is unambiguous.

- **Mechanism:** A Bloom Filter is a space-efficient probabilistic data structure that tests whether an element is a member of a set. It yields no false negatives and minimal false positives.
- **Application:** Create a Bloom Filter for the "Programming" domain containing reserved keywords like `def`, `public static void`, `struct`, `import`, `console.log`.
- **Advantage:** This check is essentially free ($O(k)$ where k is the number of hash functions, usually 1-3). It allows the system to snap into "Code Mode" the instant a user says "function" or "variable," providing a highly responsive feel.

2.2 Level 2: Lightweight Static Embeddings (Model2Vec)

If the statistical gatekeeper returns ambiguous results (e.g., the user says "The operation was successful" – is this medical, military, or business?), the system elevates the text to Level 2. Here, we require **semantic understanding**, but standard Transformers are too slow.

2.2.1 The Bottleneck of Transformers

Standard BERT-based models (like `bert-base-uncased`) typically have ~110 million parameters. Even distilled versions (DistilBERT) have ~66 million.² Running inference on these involves millions of matrix multiplications.

- **Latency:** On a CPU, a single inference of DistilBERT might take **50ms - 100ms**.³ While strictly within the 200ms budget, this consumes 25-50% of the available time, leaving little room for the other 5 stages of the pipeline.

2.2.2 The Solution: Model2Vec

Recent research (2024-2025) has introduced **Model2Vec**, a technique to distill the knowledge of a large Transformer (like BERT or RoBERTa) into a small, static embedding model.¹

- **Mechanism:** Model2Vec pre-computes the output embeddings of the Transformer for the entire vocabulary. It effectively creates a "super-powered Word2Vec" lookup table. During inference, it simply looks up the vector for each word and averages them (or uses a weighted average).
- **Performance:**
 - **Speed:** Inference is up to **500x faster** than the original Transformer. We are talking about **1-3ms** for a 50-word buffer.¹
 - **Size:** The model is tiny (tens of MBs), fitting easily into CPU cache.
 - **Accuracy:** It retains much of the semantic clustering capability of the parent model. It knows that "scalpel" and "incision" are related, even if they never appear together in the training set.

Recommendation: Use a pre-trained Model2Vec model (e.g., potion-base-8M) as the primary Level 2 classifier. It offers the best balance of semantic depth and raw speed.

2.3 Level 3: Zero-Shot Classification via SetFit

The user's requirement includes **Zero-Shot** detection: the ability to detect a domain (e.g., "Geology") without having explicitly trained a model on Geology texts. This is crucial for a local app where users might want to define custom profiles.

2.3.1 SetFit (Sentence Transformer Fine-tuning)

SetFit is a framework that adapts Sentence Transformers for classification tasks using contrastive learning.⁶ It is particularly adept at few-shot and zero-shot scenarios.

- **Zero-Shot Logic:** Instead of training a classifier head with classes 0, 1, 2, we use the embedding space directly.
 1. **Define Prototypes:** The user (or system) defines the domain via a description: "Texts about rocks, minerals, and earth processes."
 2. **Embed Prototypes:** The system computes the embedding vector for this description using the SetFit model. This becomes the "Centroid" for the Geology domain.
 3. **Inference:** When new text arrives, embed it and calculate the cosine distance to the Geology Centroid. If it is close, classify it as Geology.

2.3.2 ONNX Runtime and Quantization

To run SetFit efficiently on Windows:

- **ONNX:** The model must be exported to the **Open Neural Network Exchange (ONNX)** format. This allows the use of **Microsoft.ML.OnnxRuntime**, a highly optimized inference engine for Windows.⁴
- **Quantization:** We apply **Dynamic Quantization** (Float32 → Int8). This reduces the

model size by 4x and speeds up CPU inference significantly (often 2-3x) by using integer arithmetic instead of floating-point.

- *Latency Impact:* A quantized MiniLM-L6 (often used as a backbone for SetFit) can achieve **sub-5ms inference** on modern CPUs.³

2.4 Generating Training Data with Local LLMs

A novel "Local-First" approach to Zero-Shot is to use a local LLM to *create* the training data on the fly.

1. **User Action:** User creates a "Geology" profile.
2. **System Action:** WhisperLocal (in the background) calls a local instance of **Ollama** (running Llama 3 or Phi-3).
 - *Prompt:* "Generate 50 diverse sentences that a geologist might dictate in a field report. Include technical terms."
3. **Training:** The system uses these 50 synthetic sentences to train a lightweight SetFit classifier head.⁶
4. **Result:** The system now has a fully trained, high-accuracy "Geology" detector without the user typing a single training example. This is "Zero-Shot" from the user's perspective, but "Few-Shot" (and thus higher accuracy) from the model's perspective.

2.5 Comparative Analysis of Detection Approaches

Feature	TF-IDF / Naive Bayes	Static Embeddings (Model2Vec)	Quantized SetFit (ONNX)	Large LLM (Llama 3)
Inference Latency	< ms (Microseconds)	1 – ms	5 – ms	200 – ms
Semantic Depth	None (Keyword matching)	Medium (Word-level context)	High (Sentence-level context)	Very High (Reasoning)
Zero-Shot Ability	Poor (Requires keywords)	Medium (Vector similarity)	Excellent (Contrastive)	Native
Memory Footprint	Variable (Vocab size)	Low (~50 MB)	Medium (~100 MB)	High (4GB+)

Implementation	C# HashSet / SparseVector	Python bridge or C# port	Microsoft.ML. OnnxRuntime	Ollama / REST API
Recommended Role	Stage 1 Gatekeeper	Stage 2 Default	Stage 3 / Ambiguity Solver	Offline Teacher

Architecture Decision: The Cascaded approach is the only one that satisfies the <200ms requirement while maintaining high accuracy. The pipeline attempts Stage 1 -> Stage 2 -> Stage 3, stopping as soon as a confidence threshold is met.

3. Building Domain Correction Dictionaries at Scale

Once the domain is detected (e.g., Medical), WhisperLocal must apply a "Sanitization Layer." This involves correcting the raw phonetic output of Whisper (e.g., "my owe cardial") into the correct domain terminology ("myocardial"). Building these rules manually is the current bottleneck.

We propose three scalable strategies: **Ontology Ingestion**, **The "Bag of Hallucinations,"** and **LLM-Driven Rule Generation**.

3.1 Ontology-Based Dictionary Mining

The most robust source of truth for domain terminology is not a list of regexes, but structured **Ontologies**. These are massive, curated databases of concepts and terms.

3.1.1 Medical: SNOMED CT and RxNorm

For the Medical profile, manually adding drug names is futile. There are thousands.

- **SNOMED CT (Systematized Nomenclature of Medicine -- Clinical Terms):** This is the global standard for clinical terminology. It contains over 350,000 concepts.
 - *Data Structure:* It is distributed as a set of relational tables (Concepts, Descriptions, Relationships).
 - *Ingestion Strategy:* WhisperLocal should include a parser for the **RF2** (Release Format 2) files. We are interested specifically in the **Descriptions** table, which maps a ConceptID to multiple text representations (Synonyms).
 - *Action:* Extract all "Preferred Terms" and "Synonyms." Build a massive Trie (Prefix Tree) or Aho-Corasick automaton.
- **RxNorm:** Maintained by the National Library of Medicine (NLM), this provides normalized names for clinical drugs.
 - *Utility:* It links brand names ("Tylenol") to generic names ("acetaminophen"). This

allows WhisperLocal not just to correct spelling, but to standardize terminology if the user desires (e.g., "Convert all brand names to generics").

- **Research Validation:** Papers like **United-MedASR**¹⁰ demonstrate that augmenting ASR with these specific vocabularies significantly reduces Word Error Rate (WER) on medical entities.

3.1.2 Legal: Open Legal Dictionaries

- **Source:** The **Open Legal Dictionary** project¹¹ and **Black's Law Dictionary** (older editions in public domain) provide structured definitions.
- **Data Format:** Often available as JSON or CSV.¹²
- **Ingestion:** Similar to medical, we extract terms like "habeas corpus," "res ipsa loquitur," and "certiorari." These Latin terms are frequent failure points for Whisper, which often transliterates them as English nonsense (e.g., "rest it's a locator").

3.1.3 Technical: Mining Package Repositories

For the "Code Mode" profile:

- **Method:** Scrape the top 5,000 package names from PyPI (Python), npm (JavaScript), and NuGet (.NET).
- **Correction Logic:** If Whisper hears "num pie", and the domain is Python, the correction numpy is statistically almost certain. This data is available via public datasets on Kaggle or Google BigQuery.¹⁴

3.2 The "Bag of Hallucinations" (BoH)

Whisper has a specific pathology: when it encounters silence, background noise, or music, it tends to "hallucinate." It generates text that isn't there, often reciting phrases from its training data (e.g., YouTube subtitles).

- **The Phenomenon:** Research¹⁶ identifies a specific set of phrases that recur with high frequency in non-speech segments.
 - "Thanks for watching"
 - "Subtitles by Amara.org"
 - "MBC"
 - "999"
 - Repeating the last word 10+ times (the "looping" error).
- **The "BoH" Dictionary:** We must construct a global "Bag of Hallucinations" (BoH) blacklist.
- **Filtering Algorithm:**
 1. **Exact Match:** If a transcript segment matches a BoH phrase exactly, discard it.
 2. **Probability Check:** Hallucinations often have a distinct log-probability signature. The whisper.cpp API returns token probabilities. If a segment corresponds to a BoH phrase *and* has a suspect probability profile (either extremely high confidence on

nonsense or low confidence), it is purged.¹⁶

3. **Repetition Suppression:** A simple heuristic: if a 3-gram repeats more than 3 times in a 10-second window, truncate the repetition. This is a vital "safety valve" for the regex engine.¹⁸

3.3 The "Offline Teacher": LLM-Generated Rules

We cannot run an LLM on every sentence in <200ms. However, we can use an LLM *asynchronously* to write the regex rules that the fast engine uses. This is the **Teacher-Student** model.

- **The Concept:** The User provides a corpus (e.g., "Here are my last 10 medical reports"). The Local LLM (Ollama running Llama 3) reads them and generates the "Correction Profile."
- **Prompt Engineering for Rule Generation:**
 - Generating regex directly from an LLM is error-prone. It is better to ask for *pairs* and compile the regex deterministically.
 - **Prompt:** "You are an expert medical transcriptionist. Read the following text. Identify technical terms that a layperson or robot might mishear. For each term, provide the Correct Term and 3 likely Phonetic Misinterpretations. Return the result as a JSON array."
Example Output: [{"term": "ischemia", "mishearings": ["is scheme ia", "is key mia", "ice cream ia"]}]"
- **The Compiler:** A C# script takes this JSON. It uses a phonetic distance algorithm (Levenshtein) to verify that the "mishearing" is indeed close to the "term." If valid, it generates the regex: `\b(is\s?scheme\s?ia|is\s?key\s?mia)\b -> ischemia`.
- **Advantage:** This allows the user to build a highly robust, personalized correction dictionary simply by feeding the system their past work. The LLM does the hard work of anticipating errors.¹⁹

3.4 Community Approaches

- **Format:** To scale this beyond a single user, WhisperLocal should adopt a standard exchange format (e.g., **JSON-LD** or **YAML**) for profiles.
- **GitHub Model:** Users can host "Profile Repos" (e.g., "whisper-local-radiology"). The app can git pull these definitions.
- **Conflict Resolution:** If the "Radiology" profile wants to correct "cat" to "CT" (Computed Tomography), but the "Veterinary" profile wants to keep "cat," the **Domain Detector** (Section 2) acts as the arbiter.

4. Adaptive/Learning Corrections (The "Smart" Loop)

Static dictionaries are brittle. A true "AI" assistant learns from its mistakes. If the user

consistently corrects "weather" to "whether" in a specific context, the system must automate this.

4.1 Learning from User Edits (Levenshtein Feedback)

The most valuable signal in the system is the **User Edit**. When the user changes the text produced by WhisperLocal, they are providing a labeled training example.

- **The Algorithm:**

1. **Capture:** Store the ASR Output (T_{asr}) and the Final Saved Text (T_{final}).
2. **Diffing:** Compute the **Levenshtein Distance** (Edit Distance) between them.²¹
3. **Alignment:** Perform a "backtrace" on the edit matrix to identify specific substitutions.
 - *Example:* T_{asr} = "He has a fever." T_{final} = "He has Afib."
 - *extracted Pair:* a fever $\rightarrow$ Afib.
4. **Contextualization:** Record the surrounding words (context window). has [a fever -> Afib].
5. **Confidence Accumulator:** A single edit might be a fluke. We use a **Counter**.
 - If count(a fever -> Afib) > 3, promote this rule to the user's active dictionary.
 - This prevents "learning" typos (e.g., if the user accidentally types "Afib" as "Afibb", we don't want to learn that rule).

4.2 Phonetic Indexing with Double Metaphone

When the user dictates a word that Whisper doesn't know (e.g., a surname "Smythe" or a company "Zyloroph"), Whisper will transcribe it phonetically (e.g., "Smith" or "Xylophone"). To bridge this gap, we need a **Phonetic Index**.

- **Why Soundex Fails:** The traditional Soundex algorithm is too coarse. It maps "Smith" and "Smythe" to S530, but it also maps unrelated words together. It fails completely on non-English names.
- **The Solution: Double Metaphone:** This algorithm produces two codes for each word: a Primary (most likely English pronunciation) and a Secondary (alternate/foreign pronunciation).
 - *Example:* "Schmidt" $\rightarrow$ Primary: XMT (as in "Sh"), Secondary: SMT (as in "Sm").
- **The Workflow:**
 1. **Index:** User adds "Zyloroph" to their Personal Vocabulary. System computes $\text{DoubleMetaphone}(\text{"Zyloroph"}) \rightarrow \text{SLRF}$.
 2. **Lookup:** Whisper outputs "Zai lorf". System computes $\text{DoubleMetaphone}(\text{"Zai lorf"}) \rightarrow \text{SLRF}$.
 3. **Match:** The phonetic codes match! The system calculates the Levenshtein distance

between the *codes* and the *text*. Since the phonetic match is strong, the system swaps "Zai lorf" for "Zylorph".²²

- **Implementation:** Use the C# Phonix or Akin libraries.²¹ These are lightweight, strictly local dependencies.

4.3 Federated N-Gram Language Models

For deeper personalization, we can train a **Language Model (LM)** on the user's data. Since we cannot retrain Whisper locally, we use a lightweight **N-Gram Model** to "rescore" or autocorrect the output.

- **Federated Learning Concept:** The training data (user's documents) never leaves the machine. The model is trained on the device.²⁶
- **KenLM:** The industry standard for efficient N-gram query is **KenLM**.²⁷ It uses highly optimized data structures (Probing Hash Tables) to store millions of N-grams in minimal RAM.
- **Application:**
 - Whisper output: "I read the *red* book." vs "I read the *read* book."
 - If the user frequently writes about "read operations" (coding context), the local 3-gram model will assign a higher probability to "read book" (as in "read operation on book object") than the general model.
 - The system uses this probability to select the best candidate.

5. Context-Window Approaches (Managing the Stream)

A major flaw in naive ASR implementations is the "Context Gap." Whisper processes audio in 30-second chunks. If a sentence or entity spans the boundary between Chunk A and Chunk B, it is often transcribed incorrectly because the model lacks the full context.

5.1 The "Mind the Gap" Strategy: Overlapping Semantic Windows

We must abandon the idea of processing strict $[0 - 30s]$, $[30 - 60s]$ chunks. Instead, we implement a **Sliding Window with Overlap**.

- **The Algorithm:**
 1. **Window A:** Process $[0 - 30s]$.
 2. **Window B:** Process $[25s - 55s]$. (Note the 5s overlap).
 3. **Stitching:** We now have two versions of the audio from $25s - 30s$.
 - *Version A:* From the *end* of Window A.

- *Version B*: From the start of Window B.
- 4. **The "Center Trust" Principle**: ASR models are most accurate in the middle of their context window. The end of Window A is prone to "cut-off" errors. The start of Window B is prone to "warm-up" errors (lack of left context).
- 5. **Reconciliation**: Recent research ²⁸ suggests prioritizing the *right-hand* chunk for entities that span the boundary.
- 6. **Implementation**: Align the text from the overlapping region using the **Smith-Waterman algorithm** (local sequence alignment) to find the optimal splice point.

5.2 Topic Tracking with Exponential Decay

To prevent the "Domain Detector" from flickering wildly (e.g., switching to "Sports" because the doctor said "running a test"), we need a stateful **Topic Context**.

- **The Topic Vector**: Let V_t be the semantic vector of the current utterance (from Model2Vec/SetFit).
- **The Global Context State**: Let C_t be the running context of the session.
- **Exponential Decay Update Rule**:

$$C_t = \alpha \cdot V_t + (1 - \alpha) \cdot C_{t-1}$$

- α (alpha) is the "Learning Rate" or "Decay Factor" (e.g., 0.1).
- **Interpretation**: This formula ensures that the Global Context (C_t) is heavily weighted by the history (C_{t-1}), giving it "inertia." A single outlier sentence (V_t) will shift the context slightly, but not enough to cross the decision boundary into a new profile. Only sustained discussion in a new domain will shift the C_t enough to trigger a profile switch.³⁰
- **User Benefit**: This provides a stable, "sticky" experience where the system understands that "The patient is running" is still a medical statement, not a sports one.

6. Engineering the Local Stack (Windows Implementation)

To convert this research into a shipping Windows application, specific technology choices must be made. The constraint is **100% Local, C#/ .NET 8**.

6.1 The Technology Stack

- **Core Application**: C# / .NET 8 (WPF or WinUI 3 for the interface).

- **ASR Inference:** whisper.cpp (loaded as a native DLL via P/Invoke). This is essential for AVX2 optimization.
- **Post-Processing Inference: Microsoft.ML.OnnxRuntime.**
 - This library allows running the Quantized SetFit (Domain Detection) and Model2Vec models efficiently on the CPU.⁴
 - It supports INT8 quantization out of the box, critical for the <200ms budget.
- **Vector Database: Faiss** (Facebook AI Similarity Search) via C# bindings, or **USearch** (a smaller, header-only C++ library with C# bindings).
 - Used to store the "Centroids" for domain detection and the Phonetic Index for personal vocabulary.
- **Local LLM Interface: Ollama.**
 - WhisperLocal should invoke ollama.exe as a subprocess for the asynchronous "Offline Teacher" tasks (rule generation).
- **Database: SQLite.**
 - Stores the Ontologies (SNOMED, etc.), User Edit History, and BoH lists.

6.2 Comparison of Implementation Approaches (Ranked)

Feature	Approach A (Simple/Fast)	Approach B (Advanced/Robust)	Recommendation
Domain Detection	TF-IDF + Keyword List	Cascaded: Keyword Gate -> Quantized SetFit (ONNX)	Approach B (Best balance of speed/accuracy)
Correction	Regex List (Manual)	Ontology Ingestion + LLM-Generated Rules	Approach B (Scalable & Low Friction)
Adaptation	Manual "Add Word"	Levenshtein Feedback Loop + Phonetic Index	Approach B (True "Learning" behavior)
Context	Single Chunk (Current)	Overlapping Window + Exponential Decay Vector	Approach B (Eliminates boundary errors)

6.3 Performance Profiling

To ensure the <200ms budget is met with Approach B:

1. **Parallelization:** Run the Domain Detection (SetFit) and the Phonetic Indexing (Double Metaphone) in parallel tasks immediately after the text is received.
2. **Memory Mapping:** Use Memory-Mapped Files for the large Ontology databases (SQLite) to avoid loading 500MB of SNOMED data into RAM.
3. **SIMD:** Ensure the C# build enables SIMD (Single Instruction, Multiple Data) specifically for the Vector Dot Products in the TF-IDF and Embedding comparison stages.

7. Conclusion

The "WhisperLocal" project represents the next generation of local speech recognition. By moving beyond simple transcription and integrating a sophisticated, layered post-processing pipeline, it addresses the critical usability gaps of raw ASR.

The research confirms that a **Cascaded Architecture**—using TF-IDF for speed, Quantized Embeddings for smarts, and Offline LLMs for teaching—is the only viable path to meet the conflicting requirements of <200ms latency and high semantic accuracy on local hardware. Furthermore, by anchoring corrections in established Ontologies (SNOMED, Black's Law) and adapting via Levenshtein-based user feedback, the system evolves from a static tool into a dynamic, personalized assistant. This architecture not only solves the immediate "last mile" formatting problems but lays the groundwork for a truly privacy-first, intelligent dictation interface.

Works cited

1. Scikit-learn vs. Static Embeddings for Text Classification on Google Colab CPU!, accessed February 10, 2026, <https://moshewasserblat.medium.com/scikit-learn-up-to-x6-faster-than-static-embedding-1659680e44ca>
2. Tiny Language Models for Automation and Control: Overview, Potential Applications, and Future Research Directions - MDPI, accessed February 10, 2026, <https://www.mdpi.com/1424-8220/25/5/1318>
3. Efficiently run SetFit Models with Optimum - Hugging Face, accessed February 10, 2026, <https://huggingface.co/docs/setfit/main/tutorials/onnx>
4. Accelerate NLP inference with ONNX Runtime on AWS Graviton processors, accessed February 10, 2026, <https://aws.amazon.com/blogs/machine-learning/accelerate-nlp-inference-with-onnx-runtime-on-aws-graviton-processors/>
5. MinishLab/model2vec: Fast State-of-the-Art Static Embeddings - GitHub, accessed February 10, 2026, <https://github.com/MinishLab/model2vec>
6. Zero-shot Text Classification - Hugging Face, accessed February 10, 2026, https://huggingface.co/docs/setfit/tutorials/zero_shot
7. Compressing SetFit Models with Knowledge Distillation and Quantization - Colab,

- accessed February 10, 2026,
<https://colab.research.google.com/github/huggingface/workshops/blob/main/few-shot-learning-in-production/setfit-optimisation.ipynb>
8. Reducing CPU usage in Machine Learning model inference with ..., accessed February 10, 2026,
<https://inworld.ai/blog/reducing-cpu-usage-in-machine-learning-model-inference-with-onnx-runtime>
 9. ZERO Shot Classification w/ synthetic Data Set added (SBERT 47) - YouTube, accessed February 10, 2026, <https://www.youtube.com/watch?v=WQtQ4yvVJ7U>
 10. High-Precision Medical Speech Recognition Through Synthetic Data and Semantic Correction: United-MedASR - arXiv, accessed February 10, 2026,
<https://arxiv.org/html/2412.00055v1>
 11. digitallawyer/openlegaldictionary: An open source legal ... - GitHub, accessed February 10, 2026, <https://github.com/digitallawyer/openlegaldictionary>
 12. Open Data Glossary, accessed February 10, 2026,
<https://data.ca.gov/pages/open-data-glossary>
 13. 6. Glossary · OPEN DATA HANDBOOK, accessed February 10, 2026,
<https://chhsdata.github.io/opendatahandbook/glossary/>
 14. List of datasets for machine-learning research - Wikipedia, accessed February 10, 2026,
https://en.wikipedia.org/wiki/List_of_datasets_for_machine-learning_research
 15. Programming Language Database | Kaggle, accessed February 10, 2026,
<https://www.kaggle.com/datasets/sujaykapadnis/programming-language-database>
 16. Investigation of Whisper ASR Hallucinations Induced by Non-Speech Audio This research was supported by the National Science Centre, Poland under Grant 2021/42/E/ST7/00452, the National Centre for Research and Development, Poland under Grant INFOSTRATEG-IV/0029/2022, and by program "Excellence initiative – research university" - arXiv, accessed February 10, 2026,
<https://arxiv.org/html/2501.11378v1>
 17. Investigation of Whisper ASR Hallucinations Induced by Non-Speech Audio - ResearchGate, accessed February 10, 2026,
https://www.researchgate.net/publication/388232036_Investigation_of_Whisper_ASR_Hallucinations_Induced_by_Non-Speech_Audio
 18. A possible solution to Whisper hallucination · openai whisper · Discussion #679 - GitHub, accessed February 10, 2026,
<https://github.com/openai/whisper/discussions/679>
 19. LLM-based Generative Error Correction for Rare Words with Synthetic Data and Phonetic Context - arXiv, accessed February 10, 2026,
<https://arxiv.org/html/2505.17410v1>
 20. Improving ASR with LLM-Guided Text Generation: A Zero-Shot Approach to Error Correction, accessed February 10, 2026,
https://www.reddit.com/r/ArtificialIntelligence/comments/1hycoal/improving_asr_with_llmguided_text_generation_a/
 21. API Reference — Akin v0.2.0 - Hexdocs, accessed February 10, 2026,

- <https://hexdocs.pm/akin/>
22. Fuzzy Matching Techniques | Babel Street, accessed February 10, 2026, <https://www.babelstreet.jp/blog/fuzzy-name-matching-techniques>
 23. Find Duplicates with Phonetic Comparators - Experian Data Quality Community, accessed February 10, 2026, <https://community.experianaperture.io/discussion/758/find-duplicates-with-phonetic-comparators>
 24. Phonetic algorithms - Splink, accessed February 10, 2026, https://moj-analytical-services.github.io/splink/topic_guides/comparisons/phonetic.html
 25. eldersantos/phonix: Phonetic library for .NET - GitHub, accessed February 10, 2026, <https://github.com/eldersantos/phonix>
 26. Federated Learning of N-Gram Language Models - ACL Anthology, accessed February 10, 2026, <https://aclanthology.org/K19-1012.pdf>
 27. How To Train, Evaluate, and Fine-Tune an n-gram Language Model — NVIDIA Riva, accessed February 10, 2026, <https://docs.nvidia.com/deeplearning/riva/user-guide/docs/tutorials/asr-python-advanced-nemo-ngram-training-and-finetuning.html>
 28. [2506.22858] Mind the Gap: Entity-Preserved Context-Aware ASR Structured Transcriptions, accessed February 10, 2026, <https://arxiv.org/abs/2506.22858>
 29. (PDF) Mind the Gap: Entity-Preserved Context-Aware ASR Structured Transcriptions, accessed February 10, 2026, https://www.researchgate.net/publication/393184364_Mind_the_Gap_Entity-Preserved_Context-Aware_ASR_Structured_Transcriptions
 30. Topic Modeling for Short Texts via Optimal Transport-Based Clustering - ACL Anthology, accessed February 10, 2026, <https://aclanthology.org/2025.findings-acl.398.pdf>